

Installing R, Python, PostgreSQL, Quarto and Positron on Ubuntu 26.04

Big Data Algorithms — setup guide

These steps install the current versions of the software we use in the course on Ubuntu 26.04 LTS. They apply both to Ubuntu installed directly on your computer and to Ubuntu inside WSL on Windows (set up by following “install-wsl-ubuntu.qmd”). Mac users should follow “software_setup_mac.qmd” instead.

Run every command in the Ubuntu terminal. Commands that begin with **sudo** ask for your Ubuntu password. As of October 2026, you should end up with:

Software	Version	Check with
R	4.6	<code>R --version</code>
Python	3.14	<code>python3 --version</code>
PostgreSQL	18	<code>psql --version</code>
Quarto	1.10	<code>quarto --version</code>
Positron	2026.09	<code>Help > About</code>

Before you start

Bring Ubuntu up to date and install the download tools the steps below rely on:

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y curl wget ca-certificates
```

R

Ubuntu’s own repository carries an older R (4.5). To get the current release, we add the repository maintained by CRAN, the official home of R:

```
sudo apt install -y --no-install-recommends software-properties-common dirmngr
wget -qO- https://cloud.r-project.org/bin/linux/ubuntu/marutter_pubkey.asc | sudo tee -a /etc
sudo add-apt-repository "deb https://cloud.r-project.org/bin/linux/ubuntu $(lsb_release -cs)-
```

The last command asks you to press Enter to confirm. Now install R:

```
sudo apt install -y --no-install-recommends r-base r-base-dev
```

The `r-base-dev` package contains the compilers R needs to install add-on packages. On Ubuntu, `install.packages()` builds packages from source, so large packages can take several minutes to install.

Check the installation:

```
R --version
```

The first line should begin with `R version 4.6`.

Python

Ubuntu 26.04 ships with Python 3.14, which is the current stable series, so there is nothing new to download. We only add `pip` (the package installer) and `venv` (virtual environments):

```
sudo apt install -y python3 python3-pip python3-venv
python3 --version
```

Note that the command is `python3`, not `python`. Ubuntu maintains its own build of Python, so the last digit of the version may be slightly behind the one on `python.org`. That's fine.

Ubuntu does not allow `pip` to install packages into the system Python (you would get an `externally-managed-environment` error). Instead, create a virtual environment for the course and install packages there:

```
python3 -m venv ~/.venvs/bigdata
source ~/.venvs/bigdata/bin/activate
```

Your prompt now begins with `(bigdata)`, and inside the environment the commands `python` and `pip` refer to the environment's own copies. Type `deactivate` to leave it. You need to run the `source` line again in every new terminal where you want to use the environment. Positron looks for environments in `~/.venvs`, so it will find this one on its own.

PostgreSQL

The PostgreSQL project maintains its own repository for Ubuntu, which always has the newest release. Add it and install PostgreSQL 18:

```
sudo apt install -y postgresql-common
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh
sudo apt install -y postgresql-18
```

The second command asks you to press Enter to confirm. The database server starts on its own after installation and again every time Ubuntu starts. Check that it is running:

```
pg_lsclusters
```

You should see a line for version 18, cluster main, port 5432, with status **online**. If the status is **down**, start the server with `sudo systemctl start postgresql`.

Out of the box, only the administrative user **postgres** can connect. Create a database user with your Ubuntu username, and a database with the same name:

```
sudo -u postgres createuser --superuser $USER
createdb $USER
```

If the first command prints a warning that begins with **could not change directory to**, ignore it. The user was still created.

From now on **psql** connects without any extra options:

```
psql -c "SELECT version();"
```

This should print one row beginning with **PostgreSQL 18**. Typing **psql** alone opens the interactive SQL prompt and **\q** quits it.

Quarto

Quarto is not in any Ubuntu repository. Download the latest release from quarto.org and install it:

```
ARCH=$(dpkg --print-architecture)
wget https://quarto.org/download/latest/quarto-linux-$ARCH.deb
sudo apt install -y ./quarto-linux-$ARCH.deb
rm quarto-linux-$ARCH.deb
```

The first line detects whether your computer has an Intel/AMD processor (`amd64`) or an ARM processor (`arm64`), so that the right file is downloaded.

Check the installation:

```
quarto --version
quarto check
```

`quarto check` lists everything Quarto can find on your computer, and should show the R and Python you just installed. It may also tell you that Jupyter (for Python) and the `knitr` package (for R) are not installed yet. The next section installs `knitr`, and Jupyter is not needed for the setup in this guide.

R and Python together in Quarto

A single Quarto document can contain both R and Python code. Quarto runs the code in such a document with R's `knitr` and `rmarkdown` packages, and the `reticulate` package lets R start Python for the Python parts.

`reticulate` loads Python as a shared library, which Ubuntu keeps in a separate package. Make sure it is installed:

```
sudo apt install -y python3-dev
```

Then create your personal R package library (the folder in your home directory where R keeps the packages you install) and install the three packages:

```
Rscript -e 'dir.create(Sys.getenv("R_LIBS_USER"), recursive = TRUE, showWarnings = FALSE)'
Rscript -e 'install.packages(c("knitr", "rmarkdown", "reticulate"), repos = "https://cloud.r-project.org")'
```

The second command takes several minutes, because the packages are built from source.

Left alone, `reticulate` chooses a Python by itself, and it may not be your `bigdata` environment. Packages you installed there would then appear to be missing when you render a document. Tell `reticulate` which Python to use by setting the variable `RETICULATE_PYTHON` in `~/.Renvi`, a file R reads every time it starts:

```
echo "RETICULATE_PYTHON=$HOME/.venvs/bigdata/bin/python" >> ~/.Renviron
```

Check the result:

```
Rscript -e 'reticulate::py_config()'
```

The first line should read `python:` followed by the path of `.venvs/bigdata/bin/python` inside your home directory. A line saying `numpy: [NOT FOUND]` is expected, because nothing has been installed into the environment yet.

Positron

Positron is the editor (IDE) we use for R, Python and Quarto. It is a desktop application, so how you install it depends on where your Ubuntu lives.

Ubuntu installed directly on your computer

Open <https://positron.posit.co/download.html> in your browser, accept the license and download the “Debian/Ubuntu x64” file. If `dpkg --print-architecture` prints `arm64` on your computer, download the “Debian/Ubuntu ARM64” file instead. Then install the file you downloaded:

```
sudo apt install -y ~/Downloads/Positron-*.deb
```

Start Positron from the applications menu, or by typing `positron` in a terminal.

Ubuntu inside WSL on Windows

Positron is installed on the Windows side and connects to your Ubuntu from there. Do not install the Linux version inside WSL.

1. Open <https://positron.posit.co/download.html> in your Windows browser, accept the license, download the Windows “x64 User install” file and run it.
2. Open Positron and press Ctrl+Shift+P to bring up the Command Palette. Type “WSL” and choose the command that connects to WSL, selecting `Ubuntu-26.04` as the distribution.
3. The first connection takes a minute or two, because Positron installs a helper program inside Ubuntu. When it is done, the bottom-left corner of the window shows that you are connected to WSL.

From then on, everything Positron runs (R, Python, the terminal) runs inside Ubuntu, using the software you installed above.

Check that Positron sees R and Python

Click the interpreter selector in the top-right corner of the Positron window. It should list R 4.6 and Python 3.14, including the **bigdata** environment. Select R, and the Console at the bottom should show R's startup message. Then do the same for Python.

Positron comes with its own copy of Quarto. We installed Quarto separately so that the **quarto** command also works in the terminal.

Updating later

- R, Python and PostgreSQL are updated together with the rest of Ubuntu whenever you run `sudo apt update && sudo apt full-upgrade`.
- To update Quarto, repeat the commands in the Quarto section.
- To update Positron on Ubuntu, download the new **.deb** file and install it the same way. On Windows, Positron offers updates by itself.